

AI 协作反思日志 - Phase 3 详细设计

团队: 海底小纵队 日期: 2026年5月8日

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量 (1-5)	人工修改幅度 (%)
核心类图设计与 SOLID 缺陷注入实验	豆包	提供需求文档与初始模块划分， 要求生成包含状态/ 策略模式的完整类图；随后基于 SOLID 原则进行逐条审查与重构。	2	60%
API 规范文档生成	Gemini	编写核心业务的 RESTful API 规范， 规定全局响应格式与错误码， 并补充失败响应示例。	3	50%
数据库 ER 图与建表 SQL	ChatGPT	绘制系统 ER 图， 生成包含索引设计和隐私数据处理 （如手机号加密、密码哈希）的 MySQL 建表语句。	3	40%

2. AI 助力分析(Q1)

在本阶段，AI 极大地加速了底层代码和规范文档的产出效率。在繁琐的 Mermaid 语法编写、标准的 RESTful JSON 结构化输出，以及包含索引和约束的完整 SQL 脚本生成方面，AI 展现了强大的模板化能力。同时，AI 在数据库安全性设计上提供了专业建议（如采用不可逆哈希存储密码、对称加密存储手机号），为后续开发省去了大量查阅基础规范的时间。

3. AI 误导分析(Q2)

SOLID 检查中发现的AI设计问题: 违反了里氏替换原则（LSP）与单一职责原则（SRP）。

详细说明: AI 为了在类图中强行体现面向对象的继承特性, 让 Admin 直接继承了 User, 导致系统管理员被迫继承了 studentId (学号) 和 creditScore (校园信用分) 等毫无逻辑关联的垂直业务属性。此外, AI 初始设计的 User 类存在“上帝类”倾向, 将复杂的 login 和 register 鉴权逻辑直接塞入数据实体中。这反映出 AI 懂语法规则, 但缺乏对垂直业务领域常识的深刻理解。

4. Prompt 改进计划(Q3)

上阶段改进措施的验证结果: 上阶段在 P2 中强调了架构边界和模块化单体的约束。在此基础之上, 本阶段 AI 在生成 API 和数据库划分时, 能够较好地遵循模块边界, 未出现前后端职责混淆的现象。

本阶段新的改进措施: 在要求 AI 生成面向对象的架构设计方案时, 不能仅提供业务需求描述。必须在 Prompt 中强制要求其“区分领域数据实体 (Entity) 与业务逻辑服务 (Service)”, 并要求 AI 提前进行 SOLID 原则自查。同时, 需在提示词中显式注入业务常识 (如: 管理员不具备普通学生的业务属性), 防止其产生违背现实逻辑的代码关联。

5. 本阶段的核心工程判断

决策内容: 强行重构实体关系, 抽离独立的鉴权服务 (AuthService) 和基础账号抽象类 (BaseAccount); 在 API 设计中放弃仅依赖 HTTP 状态码, 转而建立精细化的全局业务错误码体系 (如针对并发抢单设计的 4001、评价状态校验的 4003)。

为什么 AI 无法做这个决策: AI 倾向于生成看起来最连贯、最常见的模型代码或基础的成功响应结构。它缺乏真实的工程体验, 无法预判高并发场景下的并发冲突风险, 也无法体会前端在联调时由于缺乏具体业务异常抛出而造成的开发阻塞。只有人类架构师才能基于后期维护成本、系统安全性以及前后端协作痛点, 在设计阶段果断进行架构解耦与精细化异常定义。